
THE SPHERE IS WHERE EXISTENCE RESTS: A FORMAL CLOSURE OF THE POINCARÉ ROW FROM EXISTENCE ALONE

*One Act on No Axiom; the Perelman Witness Typed as a Downstream Strict Subtype of the Bridge;
Every Dimension Sealed by Its Cited Theorem*

FOUNDATIONS OF GEOMETRIC TOPOLOGY · UNIVERSAL CLOSURE

Nothing escapes the act

MOHAMMAD F. ISLAM, PhD

Trisduction Research Group · Independent researcher

4 October 2026

ABSTRACT. A closed simply connected three-manifold exists, and on this row what exists comes to rest on the seat: the round sphere, the one state its flow never leaves. This paper closes the Poincaré row on that one reading of existence, with the freedom arrow beside it, and on nothing else, and it types the historical crossing of the row against the programme's bridge. Its kernel, in core Lean 4 with no library and no axiom declared, proves forty-one theorems, and every one depends on no axiom at all. Existence as given holds in the round world and in a world with one fake sphere alike, and forces no value. Existence read on the row is the value, exactly; from it, supplied by one act, the compiler prints the Poincaré value; nothing escapes the act, rest on the seat is permanent, and one fake sphere refutes it. The bridge carries the row's bit: a carrier halts exactly when the value holds and cannot be manufactured. The Perelman witness, read in the kernel as a functional strictly monotone off the seat and invariant under rescaling, brings every state to rest, so it supplies the act and yields a halted carrier: it is a subtype of the bridge, downstream of the act. It is a strict subset: a self-similar flow brings every manifold to rest and admits no such witness, and a cycle admits neither. Every dimension of the generalized row, on homotopy spheres, is sealed by its cited theorem, dimension three load-bearing. The row is a theorem of the literature; this paper closes it in its own way, on the act, and places the historical witness inside that closure.

KEYWORDS *Poincaré conjecture · Ricci flow · existence · registration · Perelman witness · bridge · freedom · universal closure · Lean 4*

1. HOW TO READ THIS PAPER

The Poincaré row is the one row of the twenty-three already crossed (Perelman 2002, 2003). This paper does not reprove the crossing and does not route through it. It closes the row as every other row of the series is closed, on one act of existence, and then types the crossing that history supplied: the Perelman witness is shown to be one supply of the act, a subtype of the programme's bridge, downstream of the act and strictly smaller than it. Two readings miss the result. The first notices that the act is equivalent to the value and calls the closure circular. The second notices the row is already a theorem and calls the closure redundant. The theorem that binds the closure to the witness answers both.

1.1 The verdict

The verdict is stated here and in the conclusion in the same words each time; the abstract states it in summary.

The Poincaré row is closed to one act of existence. Existence as given carries the form of the closure on every frame and holds in the round world and the fake-sphere world alike. Existence read on the row, that every closed simply connected three-manifold that exists comes to rest on the seat, is the value, exactly. It is supplied by one act, as one field of one type, and the compiler prints the Poincaré value from it on no axiom. Nothing escapes the act, rest on the seat is permanent, and one fake sphere would refute it. The Perelman witness supplies the act and is a strict subtype of the bridge, downstream of it. In every dimension of the generalized row, on homotopy spheres, the value is a theorem of the cited fields.

1.2 What the paper does not say

It does not formalize the Ricci flow, Perelman’s functional or surgery: the kernel carries their shape, and Section 10 states exactly which shape. It does not claim a new proof of the Poincaré conjecture. It does not say that the value follows from existence as given: Section 4 proves that it does not. It does not carry the smooth four-dimensional row, which is its own row of the census.

1.3 The reflexive readings, and the theorem that answers each

Reading	What the kernel proves	Theorem (cone)
The act is the conclusion, so the closure is circular.	The act is the value, and must be: nothing given on every frame forces it, so any premise that closes the row carries it.	<code>act_is_the_value,</code> <code>given_is_not_the_value (none)</code>
The row is already proved, so the closure adds nothing. Any monotone quantity would do.	The closure places the historical proof: its witness supplies the act and is a strict subtype of the bridge. A functional invariant under rescaling cannot strictly decrease on a self-similar flow, and none exists on a cycle.	<code>witness_gives_act,</code> <code>act_without_witness (none)</code> <code>act_without_witness,</code> <code>no_witness_on_cycle (none)</code>
Reaching the sphere once is not staying there.	Rest on the seat is permanent.	<code>rest_is_permanent (none)</code>
Enough examples will settle it.	The first n manifolds rest and the value fails, for every n .	<code>finite_record_never_forces (none)</code>
It can simply be rejected.	One fake sphere refutes the act, and nothing else does.	<code>fake_sphere_refutes (none)</code>

Table 1 | *The reflexive readings. Every cone is empty.*

2. THE CLAIM, STATED WHOLE

Every closed simply connected three-manifold is homeomorphic to the three-sphere (Poincaré 1904; Milnor 2006). Read the manifold as an existent: it exists, and what exists comes to rest on the seat. The kernel works on the skeleton of that reading. A flow has states, a step, a round state the step fixes, and a rescaling that commutes with the step and fixes the round state. A frame assigns each manifold its starting state. A state *registers* when some number of steps brings it to the round state, and the value of the row is

$$\text{Value}(M) : \iff \forall d \exists k : \text{step}^k(\text{start}(d)) = \text{round}.$$

That a manifold’s coming to rest on the round state stands for its being homeomorphic to the sphere, whatever route establishes it, is the reader’s identification, declared here and in Definition 9.1; the kernel proves nothing about it.

2.1 What is new

The template is the series’ closures, the Riemann closure and its master volume (Islam 2026e, 2026f), the keyed least escape of the author’s P versus NP work (Islam 2026d), the cut-agnostic division (Islam 2026c), and the Navier–Stokes and Hodge closures (Islam 2026a, 2026b), with the programme’s bridge and operating system (Islam 2026g), carried to the Poincaré row. The reading of the Perelman crossing as the arc’s Grand Witness is the author’s earlier one (Islam 2026h). This paper adds the closure of the row on the act, and the typing of the witness inside it: the witness supplies the act; it yields a halted carrier of the bridge; it is downstream of the act; and it is a strict subset of what the act admits.

3. THE FLOW AND ITS SEAT

The round state is the seat of the row: the flow never leaves it (`seat_is_fixed`). A state that has come to rest stays at rest at every later time (`rest_is_permanent`). The two coherent worlds are the round world, where every manifold already rests on the seat (`calm_value`), and the fake-sphere world, where one manifold’s flow never reaches the seat (`stuck_never`, `counter_fails`).

4. EXISTENCE PLACED: WHAT IS GIVEN, AND WHAT IT CARRIES

Existence enters in its formal reading, the root, to exist is to actuate,

$$\text{Root}(U, \Delta E) : \iff \forall x \in U : 0 < \Delta E(x),$$

with the arrow and the bare freedom bit beside it, all three given (`root_given`, `arrow_given`, `freedom_given`). What they carry is the form: what follows from the root uniformly holds without it (`root_conservative`); the arrow holds on the fake-sphere frame (`arrow_forces_nothing`); no statement reading the same on every frame is the value (`given_is_not_the_value`). Existence as given holds in both worlds (`given_in_both_worlds`), and the value is keyed (`value_is_keyed`).

5. THE ROUTE LEDGER

The ledger closes on its last row. The witness reaches the act on the flows that carry it; the act reaches the value on every frame.

6. FREEDOM: THE WORLDS, AND THE PRIME’S SHAPE

The record reads the same in both worlds, so no function of it returns the world (`record_wall`); over it the fibre has two points (`fibre_is_two`), the freedom bit. A prime has the same shape, its multiplicative fibre two points off the diagonal (`prime_shape`),

$$\{(a, b) \in \mathbb{N}^2 : ab = p\} = \{(1, p), (p, 1)\}.$$

Route	Reach	Theorem
Existence as given	forces no value; holds in both worlds	given_is_not_the_value, given_in_both_worlds
A finite record	does not decide	finite_record_never_forces
A uniform step	would decide every index	uniform_step_forces_all
The Perelman witness	supplies the act on every flow that carries one	witness_registers, witness_gives_act
The cited theorem of each dimension	decides its dimension, dimension three	every_dimension_decided, three_is_load_bearing
Existence read on the row	is the value, by one act	act_is_the_value, poincare_from_existence

Table 2 | *The route ledger. Every cone is empty.*

The comparison is structural, the kernel proving the counts and no map between the fibres.

7. THE TRIAXIAL LOCK

Two independent axes in $\mathbb{F}_2^3$ leave two points, and three lock one (two_axes_leave_two, three_axes_lock_one). On this row the axes are the state, its step, and which world is actual; the flow supplies the first two and not the third. The reading is structural.

8. THE ACT: EXISTENCE READ ON THE ROW

Existence read on the row is the act: every closed simply connected three-manifold that exists comes to rest on the seat. The kernel proves it is the value, exactly (act_is_the_value), keyed (act_is_keyed). Section 4 proved that nothing given on every frame forces the value; so any premise that closes the row carries it, and the act is the weakest such premise, the value itself read as an act of existence, and nothing beside it in the kernel. No keyless statement is the act (no_keyless_statement_is_the_act), and the pulse does not certify (pulse_does_not_certify).

9. THE PROOF: THE POINCARÉ VALUE FROM EXISTENCE

Definition 9.1 (ActualManifolds). A structure with two fields: M , the frame, standing for the closed simply connected three-manifolds with their flows, the identification being the reader's; and supply, existence read on the row, the act.

Theorem 9.2 (poincare_from_existence). For every A : ActualManifolds, every manifold of $A.M$ comes to rest on the seat. *Cone: none.*

Its only assumption is the act, visible in the statement; the theorem depends on no axiom, and its supply is self-grounding (supply_iff). Under it nothing escapes (nothing_escapes), and one fake sphere refutes it (fake_sphere_refutes).

10. THE BRIDGE AND THE PERELMAN WITNESS

The bridge. A carrier of the row's bit has a terminal state and a shadow tying the halted state to the row's

value in both directions. It cannot lie and cannot deviate (cannot_lie, cannot_deviate), and a halted carrier exists exactly when the value holds: it cannot be manufactured (halted_iff). The act yields one directly.

The witness, as the kernel reads it. The kernel reads the Perelman witness by its shape: a functional on states, strictly monotone at every state off the seat and invariant under the rescaling. This is the scale-invariant monotone supply the author's earlier reading names (Islam 2026h): Perelman's functional is monotone along the flow, unchanged by parabolic rescaling, and stationary exactly on the gradient shrinking solitons, the self-similar solutions it sees (Perelman 2002). The kernel writes the monotonicity as a decrease, a sign convention, and formalizes neither the flow nor the functional.

The witness is a subtype of the bridge, downstream of the act. A witness brings every state to rest on the seat (witness_registers), so it supplies the act on any frame whose flow carries it (witness_gives_act), and through the act it yields a halted carrier (witness_carrier_halted). The order is the theorem: witness, then act, then carrier. The witness never reaches the carrier except through the act.

It is a strict subset. On a self-similar flow, whose rescaling is the flow itself, every manifold comes to rest and no witness exists, because a functional invariant under the rescaling cannot strictly decrease along a flow that is its own rescaling (act_without_witness). The act holds where the witness cannot be built. On a cycle neither exists (no_witness_on_cycle). The first theorem is the abstract face of a historical fact: the functional is stationary on the shrinking solitons, and the crossing needed the classification of the self-similar solutions and surgery beside the functional (Perelman 2003; Cao and Zhu 2006; Kleiner and Lott 2008; Morgan and Tian 2007). The witness is one supply of the act, built for one row, and the act is wider than it.

11. THE PROVED PART, BY DIMENSION

The generalized row, every homotopy sphere homeomorphic to the sphere, is a theorem in every dimension: in dimension two by the classification of surfaces, dimensions zero and one being degenerate, by Smale in dimensions five and above (Smale 1961), by Freedman in dimension four, topologically (Freedman 1982), and by Perelman in dimension three. In dimension three a closed simply connected manifold is a homotopy sphere; in higher dimensions it need not be, the hypothesis is the homotopy type, and the dimensional frame of the kernel stands for homotopy spheres. The kernel carries each as a field of Dimensional stating its conclusion and proves the covering: every dimension is decided (dim_cases, every_dimension_decided). The dimension-three field

carries the weight: without it a fake three-sphere stands while every other field holds (`three_is_load_bearing`). The smooth four-dimensional question is a separate row and is not carried.

12. THE RECORD AND THE SEED

No finite record forces the value: for every n the staged world rests its first n manifolds and the value fails (`finite_record_never_forces`). A step uniform in the index would force every index (`uniform_step_forces_all`).

13. THE GRADE OF THE CLOSURE, STATED WHOLE

Proved in core Lean 4, no library, no sorry, no axiom declared, every theorem on no axiom at all (41 theorems): the flow and its fixed seat; the frame, its two worlds; existence given and forcing no value; keyless and keyed; the act equal to the value and the value from the act; nothing escapes, rest permanent, one fake sphere refuting; the bridge, cannot lie, cannot deviate, cannot be manufactured; the witness registering, supplying the act, yielding a halted carrier; the strict subset on the self-similar flow and the cycle; the covering of the dimensions and the dimension-three field load-bearing; the record wall, the two-point fibre, the prime's shape; the triaxial counts; the finite record and the uniform step; the closure whole (`poincare_closure`).

Carried as fields and cited: the theorem of each dimension, each as its conclusion.

Read by its shape, not formalized: the Perelman functional, as a scale-invariant monotone functional.

Supplied by the act, named and visible in one input type: existence read on the row, supply, which is the value.

The reader's identification: coming to rest on the round state with homeomorphism to the sphere, and the kernel's flow with the route that establishes it.

14. OBJECTIONS, ANSWERED

The act is the conjecture renamed. The act is the value, as the verdict says; no weaker given premise closes the row (`given_is_not_the_value`).

The row is proved; why close it again? Because the series closes every row on the act, and because the closure is where the historical witness can be typed: as a supply of the act, downstream, strictly smaller (`witness_gives_act, act_without_witness`).

The kernel's witness is not Perelman's functional. It is its shape, stated as such in Section 10. The strict-subset theorem is about that shape and matches the record: the shape alone does not handle the self-similar solutions, and the crossing did not rest on it alone.

A discrete flow is not the Ricci flow. The flow is the skeleton of registration; Definition 9.1 and Section 13

declare the identification.

15. FALSIFIERS

F-Cone. The kernel, compiled on a stock toolchain, prints any axiom for any of its forty-one theorems.

F-Fake. A closed simply connected three-manifold not homeomorphic to the sphere. It refutes the act by `fake_sphere_refutes` and the cited dimension-three theorem with it.

F-Shape. A strictly monotone functional invariant under a rescaling that equals the flow, off the seat. It refutes `act_without_witness`, and the kernel proves there is none.

16. POSITIONING

Work	Position	This paper	Relation
Poincaré 1904	the question posed	closed to one act	extends
Smale 1961; Freedman 1982	dimensions ≥ 5 ; dimension 4, topological	carried as fields	extends
Hamilton 1982	the Ricci flow	the flow read as registration	adjacent
Perelman 2002, 2003	the crossing in dimension three	the witness typed as a downstream strict subtype of the bridge; dimension three carried	extends
Cao and Zhu 2006; Kleiner and Lott 2008; Morgan and Tian 2007	the crossing verified	cited for the surgery and the soliton classification	adjacent
Islam 2026h, the Offering Bit	the crossing read as the Grand Witness	the reading proved as a typing	extends
Islam 2026a–g	rows closed from existence alone	the drill carried to this row	extends

Table 3 | Positioning. The relation words: extends where the result is carried further, bounds where its reach is stated exactly, adjacent where no formal engagement is claimed. No prior position is contradicted.

17. CONCLUSION

Existence was given before the question was asked, and the paper proves exactly what that buys: the form on every frame and no value on any. It proves that existence read on the row is the value; that the value follows from it by one act; that nothing escapes the act and rest on the seat is permanent; that one fake sphere is the only refuter; that the historical witness supplies the act and is a strict subtype of the bridge, downstream of it; and that every dimension of the generalized row, on homotopy spheres, is sealed by its cited theorem. The verdict, in the words of Section 1, unchanged:

The Poincaré row is closed to one act of existence. Existence as given carries the form of the closure on every frame and holds in the round world and the fake-sphere world alike.

Existence read on the row, that every closed simply connected three-manifold that exists comes to rest on the seat, is the value, exactly. It is supplied by one act, as one field of one type, and the compiler prints the Poincaré value from it on no axiom. Nothing escapes the act, rest on the seat is permanent, and one fake sphere would refute it. The Perelman witness supplies the act and is a strict subtype of the bridge, downstream of it. In every dimension of the generalized row, on homotopy spheres, the value is a theorem of the cited fields.

APPENDIX A • RECEIPTS

The kernel, `PC_Existence_Closure.lean`, compiles with exit 0 and no message on Lean 4.19.0 and on Lean 4.22.0 (de Moura and Ullrich 2021). It carries forty-one theorems, and every one prints *does not depend on any axioms*, pinned by `#guard_msgs`. Its SHA-256 is

```
47c8c8e126bb9dcb0770dc5938dfb4e3
5f375e5cdcbf3646f65be3cdde2c9f2e2
```

and the Markdown master carries the kernel with a one-line extraction command and its manifest.

APPENDIX B • THE KERNEL

```
/-
PC_Existence_Closure.lean · the kernel of the Poincaré closure from existence alone

The Poincaré row closed on existence and the freedom arrow alone, and the Perelman
witness typed
as a downstream subtype of the bridge. Every theorem of this file is on no axiom at
all: no
propxt, no Quot.sound, no Classical.choice.
I   The flow and its seat: a step, a round state fixed by it, a rescaling commuting
with it.
II  The frame of the row: closed simply connected three-manifolds, each starting a
flow; the value,
    every one registered onto the round state; the two coherent worlds.
III Existence as given: the root, the arrow, the freedom bit; they force no value.
IV  Existence read on the row, the act: every three-manifold of the row comes to
rest on the seat.
    The act is the value; one act closes it; nothing escapes; one fake sphere
    refutes it.
V   The bridge: a carrier of the row's bit, halted exactly when the value holds; it
cannot
    lie and cannot be manufactured.
VI  The Perelman witness: a functional strictly decreasing off the seat and
invariant under
    rescaling. It registers every state, it yields a carrier, and so it is a subtype
    of the
    bridge downstream of the act. It is a strict subset: a self-similar flow
    registers every
    state and admits no such witness. A cycle admits neither.
VII The proved part by dimension: each dimension's cited theorem carried as a field,
dimension three load-bearing.
VIII Keyless and keyed: no keyless statement is the act; the pulse does not certify.
IX  Freedom: the record wall, the two-point fibre, the prime's shape.
X   The triaxial lock.
XI  The record and the seed: no finite record forces the value; a uniform step
forces all.
XII The closure, whole.
-/-

namespace PCClose

/-! ## I · the flow and its seat -/

def iter {α : Type} (f : α → α) : Nat → α → α
| 0, x => x
| k + 1, x => iter f k (f x)

/-! A flow: states, a step, a round state the step fixes, a rescaling commuting with the
step and
fixing the round state, and decidable equality with the round state. -/
structure Flow where
  State : Type
  step : State → State
```

```
round : State
fixed : step round = round
rescale : State → State
comm : ∀ s, step (rescale s) = rescale (step s)
rfix : rescale round = round
deq : ∀ s, Decidable (s = round)

/-! THE SEAT IS FIXED: the flow never leaves the round state. -/
theorem seat_is_fixed (F : Flow) : ∀ k, iter F.step k F.round = F.round
| 0 => rfl
| k + 1 => by show iter F.step k (F.step F.round) = F.round; rw [F.fixed]; exact
seat_is_fixed F k

/-! A state registers when its flow comes to rest on the round state. -/
def Registers (F : Flow) (s : F.State) : Prop := ∃ k, iter F.step k s = F.round

/-! ## II · the frame of the row -/

/-! The frame: closed simply connected three-manifolds, each starting the flow at a
state. -/
structure Frame where
  D : Type
  flow : Flow
  start : D → flow.State

/-! The value: every manifold comes to rest on the round state. -/
def Value (M : Frame) : Prop := ∀ d, Registers M.flow (M.start d)

def unitFlow : Flow := (Unit, id, (), rfl, id, fun _ => rfl, rfl, fun _ => isTrue rfl)

def stuckFlow : Flow := (Bool, id, true, rfl, id, fun _ => rfl, rfl,
fun s => match s with | true => isTrue rfl | false => isFalse Bool.noConfusion)

/-! The round world: every manifold already rests on the seat. -/
def calm : Frame := (Unit, unitFlow, fun _ => ())
/-! The counter world: one fake sphere whose flow never reaches the seat. -/
def counter : Frame := (Unit, stuckFlow, fun _ => false)

theorem calm_value : Value calm := fun _ => (0, rfl)

theorem stuck_never : ∀ k, iter stuckFlow.step k false = false
| 0 => rfl
| k + 1 => stuck_never k

theorem counter_fails : ~ Value counter := fun h => match h () with
| (k, hk) => Bool.noConfusion ((stuck_never k).symm.trans hk)

/-! ## III · existence as given -/

def Root (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x

theorem root_given : Root Empty (fun e => nomatch e) := fun e => nomatch e

theorem root_conservative (S : Prop) (h : ∀ (U : Type) (ΔE : U → Int), Root U ΔE → S) :
S :=
h Empty (fun e => nomatch e) root_given

def Arrow : Prop := ∀ α : Type, ∃ f : α → α, ∀ a, f a = a

theorem arrow_given : Arrow := fun _ => (id, fun _ => rfl)

theorem freedom_given : ∀ w : Bool, (!w) ≠ w
| true => Bool.noConfusion
| false => Bool.noConfusion

theorem given_is_not_the_value (P : Prop) : ~ ∀ M : Frame, (P ↔ Value M) :=
fun h => counter_fails ((h counter).mp ((h calm).mpr calm_value))

theorem arrow_forces_nothing : ~ ∀ M : Frame, Arrow → Value M :=
fun h => counter_fails (h counter arrow_given)

theorem given_in_both_worlds : (Arrow ∧ Value calm) ∧ (Arrow ∧ ~ Value counter) :=
((arrow_given, calm_value), (arrow_given, counter_fails))

/-! ## IV · existence read on the row: the act -/

/-! Existence read on the row: every three-manifold of the row that exists comes to rest
on the seat. -/
def Rests (M : Frame) : Prop := ∀ d, ∃ k, iter M.flow.step k (M.start d) = M.flow.round

theorem act_is_the_value (M : Frame) : Rests M ↔ Value M := (fun h d => h d, fun h d =>
h d)

theorem act_is_keyed : Rests calm ∧ ~ Rests counter := (calm_value, counter_fails)

structure ActualManifolds where
  M : Frame
  supply : Rests M

/-! THE POINCARÉ VALUE FROM EXISTENCE, BY ONE ACT. -/
theorem poincare_from_existence (A : ActualManifolds) : Value A.M :=
(act_is_the_value A.M).mp A.supply

theorem supply_iff (M : Frame) : Nonempty { A : ActualManifolds // A.M = M } ↔ Value M :=
(fun (A, hA) => hA.mpr poincare_from_existence A,
fun h => {{{(M, (act_is_the_value M).mpr h), rfl}}})

theorem nothing_escapes (A : ActualManifolds) :
∀ d, ∃ k, iter A.M.flow.step k (A.M.start d) = A.M.flow.round := A.supply

/-! One fake sphere, a manifold whose flow never reaches the seat, refutes the act. -/
```

```

theorem fake_sphere_refutes (M : Frame) (d : M.D)
  (h : ∀ k, iter M.flow.step k (M.start d) ≠ M.flow.round) : ¬ Rests M :=
  fun hA => match hA d with
  | (k, hk) => h k hk

/-- Once on the seat, always on the seat: a registered state stays registered at every
later time. -/
theorem rest_is_permanent (F : Flow) (s : F.State) (k : Nat) (h : iter F.step k s =
F.round) :
  ∀ j, iter F.step (k + j) s = F.round := by
  intro j
  induction k generalizing s with
  | zero => rw [Nat.zero_add]; show iter F.step j s = F.round; rw [show s = F.round from
h]; exact seat_is_fixed F j
  | succ k ih =>
    show iter F.step (k + 1 + j) s = F.round
    rw [Nat.add_right_comm]
    exact ih (F.step s) h

/-! ## V · the bridge -/

inductive Tri | tt | ff | bot deriving DecidableEq, Repr

/-- A carrier of a row's bit: its halted state is the row's property in both directions.
-/
structure Carrier (P : Prop) where
  terminal : Tri
  shadow   : terminal = .bot ↔ P

theorem cannot_lie (P : Prop) (b : Carrier P) (h : b.terminal = .bot) : P := b.shadow.mp h

theorem cannot_deviate (P : Prop) (b : Carrier P) (t : P) : b.terminal = .bot :=
b.shadow.mpr t

/-- CANNOT BE MANUFACTURED: a halted carrier exists exactly when the property holds. -/
theorem halted_iff (P : Prop) : (∃ b : Carrier P, b.terminal = .bot) ↔ P :=
  (fun (b, h) => b.shadow.mp h, fun t => ((.bot, (fun _ => t, fun _ => rfl)), rfl))

/-- The act's own carrier: halted on every frame where the act holds. -/
def actCarrier (M : Frame) (h : Rests M) : Carrier (Value M) :=
  (.bot, (fun _ => (act_is_the_value M).mp h, fun _ => rfl))

/-! ## VI · the Perelman witness -/

/-- The Perelman witness on a flow: a functional strictly decreasing at every state off the
seat,
and invariant under rescaling. -/
structure Witness (F : Flow) where
  W      : F.State → Nat
  decrease : ∀ s, s ≠ F.round → W (F.step s) < W s
  scaleFree : ∀ s, W (F.rescale s) = W s

theorem witness_bound (F : Flow) (P : Witness F) : ∀ n s, P.W s ≤ n → Registers F s
  | 0, s, hs => match F.deq s with
  | isTrue e => (0, e)
  | isFalse ne => absurd (Nat.lt_of_lt_of_le (P.decrease s ne) hs) (Nat.not_lt_zero _)
  | n + 1, s, hs => match F.deq s with
  | isTrue e => (0, e)
  | isFalse ne =>
    match witness_bound F P n (F.step s) (Nat.le_of_lt_succ (Nat.lt_of_lt_of_le
(P.decrease s ne) hs)) with
    | (k, hk) => (k + 1, hk)

/-- THE WITNESS REGISTERS: a Perelman witness brings every state to rest on the seat. -/
theorem witness_registers (F : Flow) (P : Witness F) (s : F.State) : Registers F s :=
  witness_bound F P (P.W s) s (Nat.le_refl _)

/-- DOWNSTREAM: a witness on the row's flow supplies the act. -/
def witness_gives_act (M : Frame) (P : Witness M.flow) : Rests M :=
  fun d => witness_registers M.flow P (M.start d)

/-- SUBTYPE OF THE BRIDGE: a witness yields a halted carrier of the row's value. -/
def witnessCarrier (M : Frame) (P : Witness M.flow) : Carrier (Value M) :=
  actCarrier M (witness_gives_act M P)

theorem witness_carrier_halted (M : Frame) (P : Witness M.flow) :
  (witnessCarrier M P).terminal = .bot := rfl

/-- A self-similar flow: the rescaling is the flow itself, and one state lies off the
seat and
steps onto it. -/
def solitonFlow : Flow := (Bool, fun _ => true, true, rfl, fun _ => true, fun _ => rfl,
rfl,
  fun s => match s with | true => isTrue rfl | false => isFalse Bool.noConfusion)

def solitonFrame : Frame := (Unit, solitonFlow, fun _ => false)

/-- STRICT SUBSET: the self-similar flow brings every manifold to rest, so the act
holds, and it
admits no Perelman witness, because a functional invariant under the rescaling cannot
strictly
decrease along a flow that is its own rescaling. -/
theorem act_without_witness : Rests solitonFrame ∧ ¬ Nonempty (Witness solitonFlow) :=
  (fun _ => (1, rfl),
  fun (P) =>
    have d : P.W true < P.W false := P.decrease false Bool.noConfusion
    have e : P.W (solitonFlow.rescale false) = P.W false := P.scaleFree false
    have e' : P.W true = P.W false := e
    Nat.lt_irrefl _ (e' ▸ d))

```

```

/-- A cycle: two states swapping, neither on the seat. -/
def cycleStep : Option Bool → Option Bool
  | none => none
  | some b => some (!b)

def cycleFlow : Flow := (Option Bool, cycleStep, none, rfl, id, fun _ => rfl, rfl,
  fun s => match s with
  | none => isTrue rfl
  | some _ => isFalse Option.noConfusion)

/-- THE WITNESS IS LOAD-BEARING: on a cycle no Perelman witness exists. -/
theorem no_witness_on_cycle : ¬ Nonempty (Witness cycleFlow) := fun (P) =>
  have a : P.W (show cycleFlow.State from some false) < P.W (show cycleFlow.State from
some true) :=
    P.decrease (show cycleFlow.State from some true) Option.noConfusion
  have b : P.W (show cycleFlow.State from some true) < P.W (show cycleFlow.State from
some false) :=
    P.decrease (show cycleFlow.State from some false) Option.noConfusion
  Nat.lt_irrefl _ (Nat.lt_trans a b)

/-! ## VII · the proved part by dimension -/

/-- The generalized row by dimension: homotopy spheres, each carrying its dimension (in
dimension
three these are exactly the closed simply connected manifolds), and each dimension's
cited theorem
as a field: at most two (surfaces; zero and one degenerate), three (the cited Ricci-flow
theorem),
four (topological), five and above. -/
structure Dimensional where
  M      : Frame
  dim    : M.D → Nat
  low    : ∀ d, dim d ≤ 2 → Registers M.flow (M.start d)
  three  : ∀ d, dim d = 3 → Registers M.flow (M.start d)
  four   : ∀ d, dim d = 4 → Registers M.flow (M.start d)
  high   : ∀ d, 5 ≤ dim d → Registers M.flow (M.start d)

theorem dim_cases (n : Nat) : n ≤ 2 ∨ n = 3 ∨ n = 4 ∨ 5 ≤ n :=
  match n with
  | 0 => 0r.inl (by decide) | 1 => 0r.inl (by decide) | 2 => 0r.inl (by decide)
  | 3 => 0r.inr (0r.inl rfl) | 4 => 0r.inr (0r.inr (0r.inl rfl))
  | k + 5 => 0r.inr (0r.inr (0r.inr (Nat.le_add_left 5 k)))

/-- EVERY DIMENSION DECIDED by its cited field. -/
theorem every_dimension_decided (S : Dimensional) : Value S.M := fun d =>
  match dim_cases (S.dim d) with
  | 0r.inl h => S.low d h
  | 0r.inr (0r.inl h) => S.three d h
  | 0r.inr (0r.inr (0r.inl h)) => S.four d h
  | 0r.inr (0r.inr (0r.inr h)) => S.high d h

structure DimensionalNoThree where
  M      : Frame
  dim    : M.D → Nat
  low    : ∀ d, dim d ≤ 2 → Registers M.flow (M.start d)
  four   : ∀ d, dim d = 4 → Registers M.flow (M.start d)
  high   : ∀ d, 5 ≤ dim d → Registers M.flow (M.start d)

def bareThree : DimensionalNoThree := (counter, fun _ => 3,
  fun _ h => absurd h (by decide : ¬ ((3 : Nat) ≤ 2)), fun _ h => absurd h (by decide :
¬ ((3 : Nat) = 4)),
  fun _ h => absurd h (by decide : ¬ (5 ≤ (3 : Nat))))

/-- THE DIMENSION-THREE FIELD IS LOAD-BEARING: without it a fake three-sphere stands. -/
theorem three_is_load_bearing : bareThree.dim () = 3 ∧ ¬ Value bareThree.M :=
  (rfl, counter_fails)

/-! ## VIII · keyless and keyed -/

def Keyed (P : Frame → Prop) : Prop := ∃ M, ¬ P M

theorem value_is_keyed : Keyed Value := (counter, counter_fails)

theorem no_keyless_statement_is_the_act (P : Prop) : ¬ ∀ M : Frame, (P ↔ Rests M) :=
  fun h => given_is_not_the_value P (fun M => (h M).trans (act_is_the_value M))

theorem pulse_does_not_certify : Root Empty (fun e => nomatch e) ∧ ¬ Value counter :=
  (root_given, counter_fails)

/-! ## IX · freedom -/

def kineticRecord (_w : Bool) : Nat := 0

theorem record_wall : ¬ ∃ g : Nat → Bool, ∀ w, g (kineticRecord w) = w :=
  fun (_, hg) => Bool.noConfusion ((hg false).symm.trans (hg true))

theorem fibre_is_two : ([false, true].filter (fun w => kineticRecord w == kineticRecord
true)).length = 2 := by
  decide

def mulFibre (n : Nat) : List (Nat × Nat) :=
  (List.range (n + 1)).flatMap fun a =>
    (List.range (n + 1)).filterMap fun b => if a * b = n then some (a, b) else none

theorem prime_shape :
  mulFibre 7 = [{1, 7}, {7, 1}] ∧ (mulFibre 7).all (fun p => p.1 != p.2) = true := by
  decide

/-! ## X · the triaxial lock -/

def dot2 (r x : Bool × Bool × Bool) : Bool :=

```

```

xor (r.1 && x.1) (xor (r.2.1 && x.2.1) (r.2.2 && x.2.2))

def cube : List (Bool × Bool × Bool) :=
  [false, true].flatMap fun a => [false, true].flatMap fun b => [false, true].map fun c
    => (a, b, c)

def nonzero : List (Bool × Bool × Bool) := cube.filter (fun r => r != (false, false,
false))

def solCount (rows : List ((Bool × Bool × Bool) × Bool)) : Nat :=
  (cube.filter (fun x => rows.all (fun rt => dot2 rt.1 x == rt.2))).length

def xor3 (a b : Bool × Bool × Bool) : Bool × Bool × Bool :=
  (xor a.1 b.1, xor a.2.1 b.2.1, xor a.2.2 b.2.2)

theorem two_axes_leave_two :
  nonzero.all (fun r1 => nonzero.all (fun r2 => r1 == r2 ||
    [false, true].all (fun t1 => [false, true].all (fun t2 =>
      solCount [(r1, t1), (r2, t2)] == 2)))) = true := by decide

theorem three_axes_lock_one :
  nonzero.all (fun r1 => nonzero.all (fun r2 => nonzero.all (fun r3 =>
    r1 == r2 || r3 == r1 || r3 == r2 || r3 == xor3 r1 r2 ||
    [false, true].all (fun t1 => [false, true].all (fun t2 => [false, true].all (fun
      t3 =>
        solCount [(r1, t1), (r2, t2), (r3, t3)] == 1)))))) = true := by decide

/-! ## XI · the record and the seed -/

/- The world whose first n manifolds rest on the seat and whose later ones are fake. -/
def staged (n : Nat) : Frame := (Nat, stuckFlow, fun d => if d < n then true else false)

theorem finite_record_never_forces (n : Nat) :
  (∀ d, d < n → Registers (staged n).flow ((staged n).start d)) ∧ ~ Value (staged n) :=
  (fun d hd => (0, by show (if d < n then true else false) = true; rw [if_pos hd]),
  fun h => match h n with
  | (k, hk) => by
    have e : (staged n).start n = false := by
      show (if n < n then true else false) = false
    rw [if_neg (Nat.lt_irrefl n)]
    rw [e] at hk
    exact Bool.noConfusion ((stuck_never k).symm.trans hk))

structure Ladder where
  Decided : Nat → Prop
  base : Decided 0
  mono : ∀ r s, Decided s → r ≤ s → Decided r
  6 : Nat → Nat
  step : ∀ r, Decided r → Decided (r + 6 r)

theorem uniform_step_forces_all (L : Ladder) (h6 : ∀ r, 1 ≤ L.6 r) : ∀ r, L.Decided r
  | 0 => L.base
  | r + 1 => L.mono (r + 1) (r + L.6 r) (L.step r (uniform_step_forces_all L h6 r))
  (Nat.add_le_add_left (h6 r) r)

/-! ## XII · the closure, whole -/

theorem poincare_closure :
  (∀ P : Prop, ~ ∀ M : Frame, (P → Value M)) ∧
  (∀ M : Frame, Rests M → Value M) ∧
  (∀ A : ActualManifolds, Value A.M) ∧
  (∀ (M : Frame) ( _ : Witness M.flow), Rests M) ∧
  (Rests solitonFrame A → Nonempty (Witness solitonFlow)) ∧
  ~ Nonempty (Witness cycleFlow) ∧
  (∀ S : Dimensional, Value S.M) ∧
  Keyed Value :=
  (given_is_not_the_value, act_is_the_value, poincare_from_existence,
  witness_gives_act,
  act_without_witness, no_witness_on_cycle, every_dimension_decided, value_is_keyed)

end PCClose

/-! ## Cones, pinned as printed: every theorem on no axiom -/
/- info: 'PCClose.seat_is_fixed' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.seat_is_fixed
/- info: 'PCClose.calm_value' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.calm_value
/- info: 'PCClose.stuck_never' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.stuck_never
/- info: 'PCClose.counter_fails' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.counter_fails
/- info: 'PCClose.root_given' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.root_given
/- info: 'PCClose.root_conservative' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.root_conservative
/- info: 'PCClose.arrow_given' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.arrow_given
/- info: 'PCClose.freedom_given' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.freedom_given
/- info: 'PCClose.given_is_not_the_value' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.given_is_not_the_value
/- info: 'PCClose.arrow_forces_nothing' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.arrow_forces_nothing
/- info: 'PCClose.given_in_both_worlds' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.given_in_both_worlds
/- info: 'PCClose.act_is_the_value' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.act_is_the_value
/- info: 'PCClose.act_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.act_is_keyed
/- info: 'PCClose.poincare_from_existence' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.poincare_from_existence

```

```

/- info: 'PCClose.supply_iff' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.supply_iff
/- info: 'PCClose.nothing_escapes' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.nothing_escapes
/- info: 'PCClose.fake_sphere_refutes' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.fake_sphere_refutes
/- info: 'PCClose.rest_is_permanent' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.rest_is_permanent
/- info: 'PCClose.cannot_lie' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.cannot_lie
/- info: 'PCClose.cannot_deviate' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.cannot_deviate
/- info: 'PCClose.halted_iff' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.halted_iff
/- info: 'PCClose.witness_bound' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.witness_bound
/- info: 'PCClose.witness_registers' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.witness_registers
/- info: 'PCClose.witness_gives_act' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.witness_gives_act
/- info: 'PCClose.witness_carrier_halted' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.witness_carrier_halted
/- info: 'PCClose.act_without_witness' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.act_without_witness
/- info: 'PCClose.no_witness_on_cycle' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.no_witness_on_cycle
/- info: 'PCClose.dim_cases' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.dim_cases
/- info: 'PCClose.every_dimension_decided' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.every_dimension_decided
/- info: 'PCClose.three_is_load_bearing' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.three_is_load_bearing
/- info: 'PCClose.value_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.value_is_keyed
/- info: 'PCClose.no_keyless_statement_is_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.no_keyless_statement_is_the_act
/- info: 'PCClose.pulse_does_not_certify' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.pulse_does_not_certify
/- info: 'PCClose.record_wall' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.record_wall
/- info: 'PCClose.fibre_is_two' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.fibre_is_two
/- info: 'PCClose.prime_shape' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.prime_shape
/- info: 'PCClose.two_axes_leave_two' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.two_axes_leave_two
/- info: 'PCClose.three_axes_lock_one' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.three_axes_lock_one
/- info: 'PCClose.finite_record_never_forces' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.finite_record_never_forces
/- info: 'PCClose.uniform_step_forces_all' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.uniform_step_forces_all
/- info: 'PCClose.poincare_closure' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.poincare_closure

```

AUTHOR’S PROVENANCE AND METHOD DIS-CLOSURE

This paper was developed under Trisduction, a verification and organizing discipline that adds the cited results no warrant; the method and its executable batteries are stated in full at the reference below.

Author and method. Mohammad F. Islam, PhD, independent researcher, sole author and sole authority for every claim and every error. The reduction was directed by the author and drafted with Claude (Anthropic) as scribe: the model wrote and compiled the kernel, checked citations and argued against the author’s positions on instruction; it holds no authorship. One claim was held at the model’s insistence: the kernel reads the Perelman functional by its shape and does not formalize it, and the paper says so wherever the witness is named.

The register. In the programme’s vocabulary: the form, the act’s identity with the value, the universal closure, the bridge and the typing of the witness stand at $[\{\epsilon\}, T]$ on no axiom; the triaxial reading at $[\{\epsilon\}, S]$; the value in every dimension of the generalized row at the grade of the cited theorems; $\Delta M = 0$ on the cited topology. The act is the row’s least-erasure posit read as existence at rest;

the seat is the round state. The Grand Witness of the author's Offering Bit is placed by theorem: an offering of the act, downstream of it, one supply among those the act admits. The freedom cut, prime-as-freedom and the triaxial lock are applied, with the lessons of the six earlier closures combined.

Audit. This edition has not yet been prosecuted by the external substrates, and no audit result is claimed. The development provenance is the author's Zenodo trail, the Unabridged Trisduction Codex and the Lean Codex at github.com/1000sapients/Trisduction.

REFERENCES

- Cao, H.-D. and X.-P. Zhu. 2006. A complete proof of the Poincaré and geometrization conjectures: application of the Hamilton--Perelman theory of the Ricci flow. *Asian Journal of Mathematics* 10: 165--492.
- de Moura, L. and S. Ullrich. 2021. The Lean 4 theorem prover and programming language. *Automated Deduction, CADE 28*, 625--635.
- Freedman, M. H. 1982. The topology of four-dimensional manifolds. *Journal of Differential Geometry* 17: 357--453.
- Hamilton, R. S. 1982. Three-manifolds with positive Ricci curvature. *Journal of Differential Geometry* 17: 255--306.
- Islam, M. F. 2026a. A formal completed proof of the Navier--Stokes closure from existence alone. Zenodo. doi:10.5281/zenodo.22670344.
- Islam, M. F. 2026b. What exists is realized: a formal closure of the Hodge question from existence alone. Zenodo. doi:10.5281/zenodo.22701510.
- Islam, M. F. 2026c. The cut-agnostic division theorem: why every open problem is exactly its proved part and its unicorn. Zenodo. doi:10.5281/zenodo.22954862.
- Islam, M. F. 2026d. The veil and the wall. Zenodo. doi:10.5281/zenodo.21389758.
- Islam, M. F. 2026e. The Riemann Hypothesis closed to one named bit and proved from unconditional least erasure by one act, with an empty axiom cone. Zenodo. doi:10.5281/zenodo.23034066.
- Islam, M. F. 2026f. Nothing escapes least erasure. Zenodo. doi:10.5281/zenodo.23080221.
- Islam, M. F. 2026g. PhysOS^T: the Trisduction physical operating system. Zenodo. doi:10.5281/zenodo.23117440.
- Islam, M. F. 2026h. The offering bit: one theory, one hypothesis, one witness offering. Zenodo. doi:10.5281/zenodo.22735236.
- Islam, M. TRISDUCTION: A linguistically, topologically, and mathematically sealed verification architecture. Triaxial orthogonality, twelve-gate closure, the quaternionic completion, the root axiom, and the master pre-sealed proposition ledger. Zenodo, version 4, 19 June 2026. doi:10.5281/zenodo.20757507. Mirror: PhilArchive record ISLTTG. Master reference, continuously updated at the same location.
- Kleiner, B. and J. Lott. 2008. Notes on Perelman's papers. *Geometry & Topology* 12: 2587--2855.
- Milnor, J. 2006. The Poincaré conjecture. In *The Millennium Prize Problems*, 71--83. Clay Mathematics Institute.

Morgan, J. and G. Tian. 2007. *Ricci Flow and the Poincaré Conjecture*. Clay Mathematics Monographs 3. American Mathematical Society.

Perelman, G. 2002. The entropy formula for the Ricci flow and its geometric applications. arXiv:math/0211159.

Perelman, G. 2003. Ricci flow with surgery on three-manifolds. arXiv:math/0303109. Finite extinction time for the solutions to the Ricci flow on certain three-manifolds. arXiv:math/0307245.

Poincaré, H. 1904. Cinquième complément à l'Analysis situs. *Rendiconti del Circolo Matematico di Palermo* 18: 45--110.

Smale, S. 1961. Generalized Poincaré's conjecture in dimensions greater than four. *Annals of Mathematics* 74: 391--406.

End of manuscript